

Predicting Company Profit Using Regression Algorithms

Table of Content

- Abstract
- Introduction
- Existing Methods
- Proposed Method with Architecture
- Methodology
- Implementation
- Conclusion

Abstract

This project report outlines the development of a machine learning model to predict the profit of a company based on its R&D Spend, Administration Cost, and Marketing Spend. The dataset comprises 50 companies, and various regression algorithms are employed to find the best predictive model. The report covers data pre-processing, model construction, evaluation, and selection of the best model based on regression metrics.

Introduction

Predicting the profit of a company is a crucial task for financial planning and decision-making. This project aims to build a machine learning model that can accurately predict profit based on R&D Spend, Administration Cost, and Marketing Spend. By leveraging regression algorithms, the model will provide valuable insights for optimizing resource allocation and maximizing profitability.

Existing Method

Existing methods for profit prediction typically involve simple linear regression or heuristic approaches. While these methods can offer some insights, they often fall short in terms of accuracy and generalizability. Machine learning models, particularly regression algorithms, provide a more robust and scalable solution for this predictive task.

Proposed Method with Architecture

The proposed method involves constructing various regression models and selecting the best one based on performance metrics. The architecture includes the following components:

- **Data Collection and Pre-processing**
 - Collecting and pre-processing the dataset to handle missing values, encode categorical variables, and normalize the data.
- **Model Construction**
 - Implementing different regression algorithms: Linear Regression, Polynomial Regression, Decision Tree Regression, Random Forest Regression, and Support Vector Regression.
- **Model Evaluation**
 - Evaluating models using regression metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), and R-squared (R^2) score.
- **Model Selection**
 - Selecting the best model based on evaluation metrics.

Methodology

1.Data Collection and Pre-processing

- **Loading the Dataset**
 - The dataset is loaded into the environment using pandas.
- **Handling Missing Values**
 - Checking for missing values and imputing or removing them as necessary.
- **Encoding Categorical Variables**
 - Converting categorical variables into numerical values using techniques such as one-hot encoding.
- **Feature Scaling**
 - Normalizing the data to ensure all features contribute equally to the model.

2. Model Construction

Task 1: Construct Different Regression Algorithms

- **Linear Regression**

- A basic model that assumes a linear relationship between independent and dependent variables.
- **Polynomial Regression**
 - Extends linear regression by considering polynomial terms.
- **Decision Tree Regression**
 - A non-linear model that splits the data into subsets based on feature values.
- **Random Forest Regression**
 - An ensemble method that combines multiple decision trees for better accuracy.
- **Support Vector Regression (SVR)**
 - Uses support vectors to perform regression, suitable for non-linear relationships.

3. Data Splitting

Task 2: Divide the Data into Train Set and Test Set

- **Splitting the Data**
 - Dividing the dataset into training and testing sets using an 80-20 split.

4. Model Evaluation

Task 3: Calculate Different Regression Metrics

- **Mean Absolute Error (MAE)**
 - Measures the average magnitude of errors in predictions.
- **Mean Squared Error (MSE)**
 - Measures the average squared difference between predicted and actual values.
- **R-squared (R^2) Score**
 - Represents the proportion of variance in the dependent variable explained by the independent variables.

5. Model Selection

Task 4: Choose the Best Model

- **Comparing Models**
 - Evaluating each model based on the above metrics and selecting the one with the best performance.

Implementation

```
# Importing libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Loading the dataset
data = pd.read_csv('50_Startups.csv')

# Data Preprocessing
x = data[['R&D Spend', 'Administration', 'Marketing Spend']].values
y = data['Profit'].values

# Splitting the dataset into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
random_state=0)

# Linear Regression
linear_regressor = LinearRegression()
linear_regressor.fit(x_train, y_train)
y_pred_linear = linear_regressor.predict(x_test)

# Polynomial Regression
poly_regressor = PolynomialFeatures(degree=2)
x_poly = poly_regressor.fit_transform(x_train)
lin_regressor2 = LinearRegression()
lin_regressor2.fit(x_poly, y_train)
y_pred_poly = lin_regressor2.predict(poly_regressor.transform(x_test))

# Decision Tree Regression
tree_regressor = DecisionTreeRegressor(random_state=0)
tree_regressor.fit(x_train, y_train)
y_pred_tree = tree_regressor.predict(x_test)

# Random Forest Regression
forest_regressor = RandomForestRegressor(n_estimators=10, random_state=0)
forest_regressor.fit(x_train, y_train)
y_pred_forest = forest_regressor.predict(x_test)

# Support Vector Regression
sc_x = StandardScaler()
sc_y = StandardScaler()
```

```

x_train_scaled = sc_x.fit_transform(x_train)
x_test_scaled = sc_x.transform(x_test)
y_train_scaled = sc_y.fit_transform(y_train.reshape(-1,1)).ravel()
svr_regressor = SVR(kernel='rbf')
svr_regressor.fit(x_train_scaled, y_train_scaled)
y_pr = svr_regressor.predict(x_test_scaled)
y_pred_svr = sc_y.inverse_transform(y_pr.reshape(-1,1)).ravel()

# Calculating Regression Metrics
def evaluate_model(y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    return mae, mse, r2

metrics = {
    'Linear Regression': evaluate_model(y_test, y_pred_linear),
    'Polynomial Regression': evaluate_model(y_test, y_pred_poly),
    'Decision Tree Regression': evaluate_model(y_test, y_pred_tree),
    'Random Forest Regression': evaluate_model(y_test, y_pred_forest),
    'Support Vector Regression': evaluate_model(y_test, y_pred_svr)
}

# Selecting the Best Model
best_model = min(metrics, key=lambda x: metrics[x][1]) # Choosing based on
lowest MSE

# Displaying Metrics
for model, (mae, mse, r2) in metrics.items():
    print(f"{model}: MAE={mae}, MSE={mse}, R2={r2}")

print(f"Best Model: {best_model}")

```

Output

```

Linear Regression: MAE=7320.44161484813, MSE=77506468.16885415,
R2=0.939395591782057
Polynomial Regression: MAE=8720.931218418915, MSE=102078137.67353782,
R2=0.9201823373990293
Decision Tree Regression: MAE=4480.289000000006, MSE=30129511.580150068,
R2=0.9764409182568788
Random Forest Regression: MAE=5975.3965999999955, MSE=50766736.70654713,
R2=0.9603041125735003
Support Vector Regression: MAE=11569.391904650076, MSE=224317169.661448,
R2=0.8246003250872043
Best Model: Decision Tree Regression

```

Conclusion

In this project, we successfully developed and evaluated multiple regression models to predict the profit of a company based on R&D Spend, Administration Cost, and Marketing Spend. The Decision Tree Regression model emerged as the best performer with the lowest MSE and highest R^2 score